

Intelligent Restaurant Management System (IRMS)

Objective:

The purpose of this assignment is to design a software architecture and implement an **Intelligent Restaurant Management System (IRMS)** following SOLID principles. The system aims to streamline restaurant operations by enabling efficient order taking, kitchen coordination, and customer service management. The goal is to create a solution that supports dynamic workflows in a restaurant environment, ensuring smooth interactions between front-of-house staff, kitchen teams, and management.

Context:

An Intelligent Restaurant Management System (IRMS) is a software application designed to optimize and automate core restaurant operations. The system relies on digital interfaces, shared databases, and software-driven logic to manage activities such as menu browsing, order placement, food preparation workflow, table status management, and inventory updates.

The IRMS coordinates communication between servers and kitchen staff, displays orders on kitchen screens, prioritizes queues, and helps ensure dishes are prepared on time. It adapts to different service conditions, such as peak hours or large group orders, making workflow adjustments that minimize delays and errors.

The system also provides management tools for tracking performance metrics, scheduling employees, updating menus, and analyzing sales trends. It serves as a centralized digital backbone that supports consistent restaurant operations across various service models.

Scope of the System:

• Digital Ordering & Menu Management

- Staff use tablets or terminals to take customer orders quickly and accurately.
- Menus can be updated in real time (e.g., marking items as unavailable).
- Orders include special instructions, allergy notes, combo selections, and customizations.

• Kitchen Order Display & Workflow Coordination

- Orders are automatically routed from front-of-house terminals to the **Kitchen Display System (KDS)**.

- KDS organizes orders by preparation time, dish category, and station (e.g., grill, fryer, dessert).
- Visual alerts notify chefs when items are nearing deadlines or when new orders arrive.
- Kitchen staff can update order progress (e.g., *Started*, *Cooking*, *Ready to Serve*).

• **Table & Reservation Management**

- Hosts manage seating charts digitally, track occupied and available tables, and estimate wait times.
- Servers can monitor each table's order status, service time, and billing status.
- Reservation module allows scheduled bookings, waitlist requests, and customer notifications.

• **Billing, Payments & Receipts**

- Integrated billing combines food, drinks, service fees, and promotions.
- Payments can be processed via cash, card, or digital wallets.
- Split-bill and tip management options are provided.

• **Inventory Monitoring**

- Staff manually update ingredient usage after prep or serving.
- Alerts notify managers when stock falls below predefined thresholds.
- Historical usage helps predict reorder quantities and manage waste.

• **Analytics & Reports**

- Sales reports show peak hours, best-selling items, and revenue performance.
- Operational analytics highlight delays, kitchen bottlenecks, and staff efficiency.
- Managers can export reports for accounting, planning, or performance reviews.

• **Administrative Tools**

- Role-based access control for managers, servers, chefs, and cashiers.
- Menu configuration, price updates, and promotional campaigns are centrally managed.
- Audit logs track critical actions such as refunds, or order cancellations.

Assignment Tasks:

Task 1: Software Architecture Design

1. **Thoroughly describe the context** of the **Intelligent Tutoring System (IRMS)** based on the basic information provided. This includes clearly identifying functional and non-functional requirements, outlining the objectives and scope of the project, etc.

2. **Create the software architecture** for the **IRMS**: The software architecture should include:
 - **Architecture Characteristics** define the success criteria of the IRMS.
 - **Structure:**
 - Compare and choose suitable architecture styles to apply to the IRMS.
 - Present the software architecture in different views including module views, component-and-connector views, and allocation views.
 - **Architecture Decisions** define the rules for how the IRMS should be constructed.
 - **Design Principles**: Guidelines for constructing the IRMS.
3. **Apply SOLID Principles**: explain how the **SOLID** principles have been applied in your design.
4. **Reflection Report**: Write a brief reflection on how applying SOLID principles helped you improve the design of the IRMS. Discuss challenges you faced and how adhering to these principles made your system more modular, maintainable, and extensible.

Task 2: Code Implementation (choose **at least one main module** to implement)

1. **Implement Core Functionalities**: Implement key system features based on your design, ensuring the code adheres to the SOLID principles.

Submission Instructions:

- Submit a detailed report containing a GitHub URL (source code) on the LMS website.